

Notatki z wykładu: Problem sumy podzbiorów
NP-zupełność i w pełni wielomianowy schemat aproksymacyjny

15 kwietnia 2026

1 Problem sumy podzbiorów

Dany jest zbiór liczb naturalnych $S = \{x_1, \dots, x_n\} \subseteq \mathbb{N}$ oraz liczba $t \in \mathbb{N}$.

Definicja 1 (wersja decyzyjna). Czy istnieje podzbiór $S' \subseteq S$ taki, że $\sum_{x_i \in S'} x_i = t$?

Definicja 2 (wersja optymalizacyjna). Znaleźć podzbiór $S' \subseteq S$ taki, że suma $\sum_{x_i \in S'} x_i$ jest *największa możliwa*, ale nie przekracza t .

Oznaczenia. Dla zbioru $P \subseteq \mathbb{N}$ i liczby $x \in \mathbb{N}$ przyjmujemy

$$P + x = \{s + x : s \in P\}.$$

Na przykład dla $P = \{1, 2, 3, 6\}$ oraz $x = 2$ mamy $P + x = \{3, 4, 5, 8\}$.

Analogicznie dla *listy* L liczb naturalnych i $x \in \mathbb{N}$ przez $L + x$ oznaczamy listę powstałą z L przez dodanie x do każdego elementu. Na przykład dla $L = (3, 5, 8)$ i $x = 3$ mamy $L + x = (6, 8, 11)$.

2 Dokładny algorytm

Lemat 1. Niech $P_0 = \{0\}$ oraz $P_i = P_{i-1} \cup (P_{i-1} + x_i)$ dla $i \in [n]$. Wtedy zbiór P_n składa się ze wszystkich możliwych sum podzbiorów zbioru S .

Dowód. Pokażemy przez indukcję po i , że P_i składa się ze wszystkich możliwych sum elementów podzbiorów zbioru $\{x_1, x_2, \dots, x_i\}$.

Baza ($i = 0$). $P_0 = \{0\}$, a jedyny podzbiór $\emptyset$ ma sumę $\sum_{x_i \in \emptyset} x_i = 0$. ✓

Krok indukcyjny. Załóżmy, że P_{i-1} składa się ze wszystkich możliwych sum elementów podzbiorów zbioru $\{x_1, \dots, x_{i-1}\}$. Niech $S' \subseteq \{x_1, \dots, x_i\}$.

- Jeśli $x_i \notin S'$, to $S' \subseteq \{x_1, \dots, x_{i-1}\}$, więc

$$\sum_{x_k \in S'} x_k \in P_{i-1} \subseteq P_i.$$

- Jeśli $x_i \in S'$, to $S' \setminus \{x_i\} \subseteq \{x_1, \dots, x_{i-1}\}$, więc

$$\sum_{x_k \in S'} x_k = \left(\sum_{x_k \in S' \setminus \{x_i\}} x_k \right) + x_i \in P_{i-1} + x_i \subseteq P_i.$$

Zatem P_i zawiera wszystkie możliwe sumy elementów z $\{x_1, \dots, x_i\}$. Nie zawiera też żadnych innych elementów, bo P_{i-1} to sumy podzbiorów $\{x_1, \dots, x_{i-1}\}$, a $P_{i-1} + x_i$ to sumy podzbiorów zawierających x_i . Dla $i = n$ otrzymujemy tezę. $\square$

Zakładamy, że dysponujemy procedurą MERGE-LISTS, która na wejściu bierze dwie *posortowane* listy liczb naturalnych L, L' i zwraca posortowaną listę elementów z $L \cup L'$ z usuniętymi duplikatami. Jej złożoność wynosi

$$\Theta(|L| + |L'|).$$

Algorithm 1 Extract-Subset-Sum(S, t)

```

1:  $n \leftarrow |S|$ 
2:  $L_0 \leftarrow (0)$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $L_i \leftarrow \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5:   usuń z  $L_i$  wszystkie liczby większe od  $t$ 
6: end for
7: return największy element listy  $L_n$ 

```

Poprawność wyniku bezpośrednio z Lematu 1.

3 Analiza złożoności

Długość listy L_i może być **wykładnicza** względem i — nawet 2^i .

Dla instancji $I = (S, t)$ rozmiar wejścia wynosi

$$|I| = \Theta\left(\log t + \sum_{i=1}^n \log x_i\right).$$

Jeśli t oraz x_i są liczbami n -cyfrowymi, to $\log t, \log x_i = \Theta(n)$, więc

$$|I| = \Theta(n + n^2) = \Theta(n^2).$$

Skoro lista L_n może mieć długość 2^n , algorytm EXTRACT-SUBSET-SUM jest *wykładniczy*.

Jeśli natomiast t jest ograniczone przez funkcję wielomianową od $n = |S|$, czyli $t = O(n^p)$ dla pewnego $p \in \mathbb{N}$, to długości list L_i są ograniczone przez t i w tym przypadku algorytm działa w czasie wielomianowym.

4 NP-zupełność problemu sumy podzbiorów

Twierdzenie 1. *Problem sumy podzbiorów jest NP-zupełny.*

Dowód.

1. Problem należy do NP. Mając daną dla (S, t) odpowiedź $S' \subseteq S$, możemy w czasie wielomianowym sprawdzić, czy $\sum_{x_i \in S'} x_i = t$.

2. Redukcja 3-SAT $\leq_p$ Subset-Sum. Niech Φ będzie formułą 3-SAT w postaci CNF, ze zmiennymi $x_1, x_2, \dots, x_n$ i klauzulami $C_1, C_2, \dots, C_m$. Bez straty ogólności zakładamy, że:

- żadna klauzula nie zawiera jednocześnie zmiennej i jej zaprzeczenia (w przeciwnym razie klauzula jest trywialnie spełniona);
- każda zmienna występuje w co najmniej jednej klauzuli.

Zbudujemy instancję (S, t) problemu SUBSET-SUM.

Konstrukcja. Wszystkie liczby mają $n + m$ cyfr. Pierwsze n cyfr etykietujemy zmiennymi, ostatnie m cyfr — klauzulami.

- Dla każdej zmiennej x_i tworzymy dwie liczby r_i oraz r'_i . Obie mają cyfrę 1 na pozycji zmiennej x_i i 0 na pozostałych pozycjach zmiennych. Ponadto:
 - jeśli literal x_i występuje w C_j , to r_i ma cyfrę 1 na pozycji C_j ;
 - jeśli literal $\overline{x_i}$ występuje w C_j , to r'_i ma cyfrę 1 na pozycji C_j .
- Dla każdej klauzuli C_j tworzymy dwie liczby uzupełniające: s_j z cyfrą 1 na pozycji C_j oraz s'_j z cyfrą 2 na pozycji C_j (0 na pozostałych pozycjach).
- Liczba docelowa t ma cyfrę 1 na każdej pozycji zmiennej oraz cyfrę 4 na każdej pozycji klauzuli:

$$t = \underbrace{11 \dots 1}_n \underbrace{44 \dots 4}_m.$$

Liczby r_i, r'_i, s_j, s'_j są parami różne (wynika to z konstrukcji i założeń o Φ). Suma cyfr na każdej pozycji wszystkich liczb wynosi co najwyżej 6, więc przy dodawaniu *nie występują przeniesienia* — każdą pozycję można analizować niezależnie.

Poprawność. ($\Rightarrow$) Jeśli Φ jest spełnialna, ustalmy wartościowanie spełniające. Do S' bierzemy r_i , gdy $x_i = 1$, oraz r'_i , gdy $x_i = 0$; wtedy każda pozycja zmiennej sumuje się do 1. Dla pozycji klauzuli C_j liczba wybranych liczb r/r' mających tam 1 jest równa liczbie spełnionych literalów w C_j , czyli należy do $\{1, 2, 3\}$. Dobierając liczby uzupełniające s_j (cyfra 1) i/lub s'_j (cyfra 2) dopełniamy każdą pozycję klauzuli do 4. Stąd suma S' jest równa t .

($\Leftarrow$) Jeśli istnieje $S' \subseteq S$ o sumie t , to dla każdej pozycji zmiennej dokładnie jedna z liczb r_i, r'_i należy do S' (bo tylko one mają tam niezerową cyfrę, a suma na tej pozycji to 1). Przyjmujemy $x_i = 1$, gdy $r_i \in S'$, oraz $x_i = 0$, gdy $r'_i \in S'$. Dla każdej pozycji klauzuli C_j liczby uzupełniające dają co najwyżej $1 + 2 = 3$, więc do osiągnięcia 4 potrzebna jest co najmniej jedna liczba r_i lub r'_i mająca 1 na pozycji C_j — czyli literal spełniający C_j . Zatem każda klauzula jest spełniona i Φ jest spełnialna.

Redukcja działa w czasie wielomianowym: konstruujemy $2(n + m)$ liczb o $n + m$ cyfrach, a ich utworzenie i posortowanie jest wielomianowe względem $n + m$. $\square$

4.1 Przykład redukcji

Rozważmy formułę o $n = 3$ zmiennych i $m = 4$ klauzulach:

$$C_1 = (x_1 \vee \overline{x_2} \vee \overline{x_3}), \quad C_2 = (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}), \quad C_3 = (\overline{x_1} \vee \overline{x_2} \vee x_3), \quad C_4 = (x_1 \vee x_2 \vee x_3),$$

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4.$$

Odpowiadająca instancja SUBSET-SUM (liczby $(n + m) = 7$ -cyfrowe):

	x_1	x_2	x_3	C_1	C_2	C_3	C_4
r_1	1	0	0	1	0	0	1
r'_1	1	0	0	0	1	1	0
r_2	0	1	0	0	0	0	1
r'_2	0	1	0	1	1	1	0
r_3	0	0	1	0	0	1	1
r'_3	0	0	1	1	1	0	0
s_1	0	0	0	1	0	0	0
s'_1	0	0	0	2	0	0	0
s_2	0	0	0	0	1	0	0
s'_2	0	0	0	0	2	0	0
s_3	0	0	0	0	0	1	0
s'_3	0	0	0	0	0	2	0
s_4	0	0	0	0	0	0	1
s'_4	0	0	0	0	0	0	2
t	1	1	1	4	4	4	4

Dla wartościowania $x_1 = 1$, $x_2 = 0$, $x_3 = 0$ (spełniającego Φ) wybieramy r_1, r'_2, r'_3 , których suma na pozycjach klauzul to $(3, 2, 1, 1)$. Dopełniając liczbami uzupełniającymi $s_1, s'_2, s_3, s'_3, s_4, s'_4$ otrzymujemy na każdej pozycji klauzuli wartość 4, więc

$$S' = \{r_1, r'_2, r'_3, s_1, s'_2, s_3, s'_3, s_4, s'_4\} \quad \text{ma sumę } t.$$

5 W pełni wielomianowy schemat aproksymacyjny

Pokażemy w pełni wielomianowy schemat aproksymacyjny (FPTAS) dla wersji optymalizacyjnej.

Pomysł. Będziemy *skracać* listy L_i . Niech $0 < \delta < 1$. Z listy L tworzymy krótszą listę L' tak, by dla każdego elementu y usuniętego z L istniał w L' element z leżący blisko niego od dołu, dokładniej:

$$\frac{y}{1+\delta} \leq z \leq y.$$

Przykład. Dla $L = (15, 16, 17, 25, 27, 28, 35, 36, 37)$ oraz $\delta = 0,1$ przycinanie daje

$$L' = (15, 17, 25, 28, 35).$$

(Np. 16 zostaje pochłonięte przez 15, bo $16 \leq 15 \cdot 1,1 = 16,5$; element 17 zostaje, bo $17 > 16,5$.)

Procedura TRIM realizuje ten pomysł dla listy $L = (y_1, \dots, y_m)$ posortowanej rosnąco i parametru $0 < \delta < 1$:

Czas działania procedury TRIM jest liniowy względem $|L|$.

Schemat aproksymacyjny powstaje z dokładnego algorytmu przez przycinanie list parametrem $\delta = \varepsilon/(2n)$ w każdej iteracji:

Algorithm 2 Trim(L, δ)

```
1:  $m \leftarrow |L|$ 
2:  $L' \leftarrow (y_1)$ 
3:  $last \leftarrow y_1$ 
4: for  $i \leftarrow 2$  to  $m$  do
5:   if  $y_i > last \cdot (1 + \delta)$  then
6:     dołącz  $y_i$  na koniec  $L'$ 
7:      $last \leftarrow y_i$ 
8:   end if
9: end for
10: return  $L'$ 
```

Algorithm 3 Approx-Subset-Sum(S, t, ε)

```
1:  $n \leftarrow |S|$ 
2:  $L_0 \leftarrow (0)$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:    $L_i \leftarrow \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5:    $L_i \leftarrow \text{TRIM}(L_i, \varepsilon/(2n))$ 
6:   usuń z  $L_i$  wszystkie liczby większe od  $t$ 
7: end for
8: return największy element listy  $L_n$ 
```
